

Avaliação da Taxa de Falhas de TLB em Traces de Execução Utilizando Algoritmos de Otimização Combinatória

Pedro Mujica

Centro de Ciências Exatas e Tecnológicas
Universidade Estadual do Oeste do Paraná
Cascavel, Brazil
pedro.mujica@unioeste.br

Matheus Moraes

Centro de Ciências Exatas e Tecnológicas
Universidade Estadual do Oeste do Paraná
Cascavel, Brazil
matheus.moraes7@unioeste.br

I. INTRODUÇÃO

A *Translation Lookaside Buffer* (TLB) é uma estrutura fundamental para o gerenciamento de memória que objetiva evitar consultas à tabela de páginas, reduzindo o custo dos acessos à memória e contribuindo para o desempenho de processos. [1].

Nesse contexto, este trabalho propõe um estudo da relação entre algoritmos de otimização combinatória e o comportamento da TLB, verificando como diferentes estratégias de busca podem influenciar os acessos realizados durante a execução dos algoritmos.

Para essa análise, foram utilizados dois problemas de otimização combinatória. Na primeira aplicação, foi utilizada a técnica de Busca Tabu para resolver o Problema de Designação Generalizada, cujo objetivo é atribuir módulos a programadores minimizando o custo total e respeitando as restrições de carga horária. No segundo, foi desenvolvido o algoritmo de Recozimento Simulado para resolver o Problema de Conexão de Circuitos, que busca estabelecer as conexões entre os componentes respeitando as restrições de quantidade de conexões de cada componente. A partir da execução dessas aplicações, serão analisados os traces de acesso à memória e suas respectivas *reference strings*, permitindo comparar o comportamento da TLB em diferentes cenários.

II. DESCRIÇÃO DO PROBLEMA

O problema abordado neste trabalho está relacionado ao custo da tradução de endereços virtuais em endereços físicos durante o acesso à memória. Em um sistema de memória paginada, cada referência à memória exige uma consulta à tabela de páginas para identificar o quadro físico e, posteriormente, um novo acesso para obter o dado ou instrução desejado. Esse processo aumenta o custo do acesso à memória, tornando necessária uma solução para reduzir esse tempo.

A TLB é um mecanismo utilizado para reduzir esse custo. Localizada na MMU (*Memory Management Unit*), ela armazena traduções de páginas utilizadas recentemente. Ao receber um endereço virtual, a MMU identifica o número da página e realiza uma busca paralela nas entradas da TLB. Quando

a página é encontrada, ocorre um *TLB hit*, permitindo obter diretamente o quadro físico sem consultar a tabela de páginas. Caso a página não seja encontrada, ocorre um *TLB miss*, sendo necessária uma consulta à tabela de páginas..

Dessa forma, o comportamento da TLB pode ser avaliado pela quantidade de *hits* e *misses* durante a execução das aplicações. Neste trabalho, essa análise será realizada a partir das *reference strings* obtidas dos *traces* de execução, considerando diferentes tamanhos de TLB e algoritmos de substituição Random e LRU (*Least Recently Used*).

III. METODOLOGIA

A. Ferramentas utilizadas

- A ferramenta Valgrind [2], com o auxílio do módulo Lackey, foi empregada na geração dos traces para registrar os acessos à memória decorrentes da execução das aplicações;
- O ambiente de desenvolvimento integrado usado foi o Visual Studio Code, em conjunto com a ferramenta de versionamento de código Git, para permitir o controle das alterações realizadas;

B. Aplicações de Otimização Combinatória

1) *Problema da Designação Generalizada*: O Problema da Designação Generalizada consiste em atribuir um conjunto de módulos a um conjunto de programadores, buscando minimizar o custo total das atribuições sem ultrapassar a carga horária disponível de cada programador. Nesse contexto, cada possível atribuição possui um custo associado e uma quantidade de horas necessária para o desenvolvimento do módulo. A aplicação desenvolvida considera 4 programadores e 8 módulos. Como entradas, são utilizadas uma matriz de custos, que representa o custo de cada programador para desenvolver cada módulo, uma matriz de carga horária, que representa o número de horas necessárias para cada atribuição, e um vetor contendo a carga horária máxima disponível para cada programador.

Para resolver o problema, foi utilizado o algoritmo de Busca Tabu. A partir de uma solução inicial, o algoritmo gera

soluções vizinhas modificando o programador responsável por determinado módulo. As novas soluções são avaliadas considerando tanto o custo total quanto as restrições de carga horária. Uma lista tabu é utilizada para impedir temporariamente determinados movimentos, evitando que o algoritmo retorne repetidamente a soluções já exploradas. Além disso, é utilizado um critério de aspiração que permite aceitar um movimento tabu quando este produz uma solução melhor que a solução atual.

2) *Problema da Conexão de Circuitos*: O Problema da Conexão de Circuitos consiste em estabelecer um conjunto de conexões entre componentes, buscando minimizar o custo total das conexões. Nesse problema, cada componente é representado por uma coordenada (x, y) , e o custo de uma conexão é determinado pela distância euclidiana entre os dois componentes conectados.

A aplicação recebe como entradas a quantidade de componentes, a quantidade total de conexões a serem estabelecidas, o número máximo de conexões permitido por componente e as coordenadas (x, y) de cada componente. A solução inicial é construída de forma a garantir que cada componente possua pelo menos uma conexão e que nenhum componente ultrapasse o limite máximo estabelecido.

Para a resolução do problema, foi utilizado o algoritmo de Recozimento Simulado (*Simulated Annealing*). O algoritmo parte de uma solução inicial e gera novas soluções por meio da substituição de conexões existentes. Uma conexão é removida, desde que sua retirada não deixe um dos componentes sem conexões, e uma nova conexão válida é adicionada. A nova solução é avaliada pelo custo total das conexões. Soluções melhores são aceitas diretamente, enquanto soluções piores podem ser aceitas probabilisticamente de acordo com a temperatura atual, permitindo ao algoritmo escapar de mínimos locais. A temperatura é reduzida progressivamente utilizando um fator de resfriamento de 0,96, até atingir a temperatura mínima definida.

3) *Geração dos Traces*: Para geração dos traces, a opção `-trace-mem=yes` foi utilizada para habilitar o registro dos acessos à memória realizados durante a execução das aplicações, gerando os arquivos com extensão `.trace` utilizados nas etapas seguintes.

A divisão das aplicações e a quantidade aproximada de acessos registrados nos respectivos *traces* é apresentada a seguir:

- `pdg.c` referente à Busca Tabu aplicada ao Problema da Designação Generalizada e com execução configurada com limite de iterações, resultando em aproximadamente 2.000.000 de acessos registrados no *trace*;
- `SimulatedAnnealing.cpp` referente ao Recozimento Simulado aplicado ao Problema da Conexão de Circuitos e com execução resultando em aproximadamente 5.000.000 de acessos registrados no *trace*.

C. Geração da Reference String e Simulador

Cada acesso a memória, representa acesso a instruções ou endereços de memória. Considerando que cada página tem

tamanho de 4096 bytes e o espaço de endereçamento possui 48 bits, logo são necessários 12 bits para o deslocamento e 36 bits para o número da página. Com essas especificações, foi gerada a Reference String que foi utilizada como entrada no simulador da TLB, esse simulador recebe como parâmetro de linha de comando o arquivo de trace (extensão `.trace`), passado via `argv`, contendo o registro dos acessos à memória a serem analisados.

A geração da reference string e a simulação da TLB são realizadas pelas seguintes etapas:

- `gerarReferenceString()` abre o arquivo de trace recebido e lê cada linha válida, extraindo o endereço em hexadecimal e o tamanho do acesso em bytes, convertendo o endereço para decimal. O número da página é obtido descartando-se os 12 bits menos significativos do endereço; como um acesso pode ultrapassar o limite de uma página, o endereço final (endereço + tamanho) também é convertido em número de página, verificando se o acesso abrange uma ou duas páginas. Por fim, páginas repetidas consecutivamente não são adicionadas novamente à reference string, reduzindo o número de entradas processadas e permitindo o cálculo da taxa de compressão, obtida pela divisão do total comprimido pelo total de acessos original.
- `simularTLB()` recebe o número de entradas da TLB e a reference string comprimida, simulando dois algoritmos de substituição de página. No algoritmo LRU (*Least Recently Used*), uma fila mantém a ordem de uso das páginas, reordenando a página acessada para o final a cada hit e substituindo, em caso de miss, a página que está há mais tempo sem ser referenciada. No algoritmo Random, em caso de miss com a TLB cheia, uma entrada é escolhida aleatoriamente entre as posições ocupadas para ser sacrificada. Para cada algoritmo são contabilizados o número de hits e misses, permitindo o cálculo da taxa de acerto (*hit rate*) de cada política de substituição.

IV. ANÁLISE E DISCUSSÃO DOS RESULTADOS

Os testes foram conduzidos considerando diferentes tamanhos de TLB, com 4, 6, 8 e 10 entradas, aplicados sobre as duas aplicações selecionadas. Para cada configuração, foram avaliados os algoritmos de substituição LRU e Random, registrando-se o número total de acessos original, a quantidade de acessos comprimida, a taxa de compressão, o número de hits e misses na TLB e a taxa de acerto correspondente. A Tabela I apresenta os resultados obtidos para o problema da conexão de circuitos e a Tabela II apresenta os resultados obtidos para o problema da Designação Generalizada com a Busca Tabu.

Tabela I
RECOZIMENTO SIMULADO NO PROBLEMA DA CONEXÃO DE CIRCUITOS

Tamanho da TLB	Algoritmos de substituição	
	LRU	Random
	Taxa de acerto	
4	91.07%	86.34%
6	93.67%	91.01%
8	94.83%	92.99%
10	95.67%	94.25%

Tabela II
BUSCA TABU NO PROBLEMA DA DESIGNAÇÃO GENERALIZADA

Tamanho da TLB	Algoritmos de substituição	
	LRU	Random
	Taxa de acerto	
4	97.71%	96.25%
6	98.77%	98.53%
8	99.77%	99.65%
10	99.81%	99.72%

A análise dos resultados evidencia uma predominância significativa de acertos na TLB em relação às falhas, especialmente no problema da Designação Generalizada, no qual a diferença entre os algoritmos de substituição LRU e Random se mostra pouco expressiva. Observa-se, ainda, que o aumento do número de entradas da TLB resulta em uma elevação da taxa de acertos para ambos os algoritmos, indicando que uma maior capacidade de armazenamento das entradas tende a reduzir a ocorrência de falhas. Verifica-se também uma diferença entre as taxas de acerto obtidas nos dois problemas analisados. Essa diferença está relacionada, entre outros fatores, à quantidade e às características dos acessos à memória realizados por cada aplicação. Os diferentes padrões de acesso e de reutilização das páginas de memória podem influenciar diretamente a localidade dos acessos e, conseqüentemente, o comportamento da TLB. Dessa forma, os resultados sugerem que tanto o tamanho da TLB quanto as características do padrão de acesso à memória exercem influência significativa sobre a taxa de acerto observada.

V. CONCLUSÕES

Os resultados obtidos permitem concluir que o aumento do número de entradas da TLB contribui para a elevação da taxa de acertos, tanto para o algoritmo LRU quanto para o Random, nos dois problemas analisados. Em todas as configurações avaliadas, o algoritmo LRU apresentou taxas de acerto superiores às obtidas pelo Random, principalmente no problema da Designação Generalizada, embora essa diferença tenha sido reduzida à medida que o tamanho da TLB aumentou. Dessa forma, dentro das configurações e aplicações consideradas neste trabalho, o LRU apresentou um desempenho superior ao Random na utilização da TLB.

Também foi observada uma diferença significativa entre os resultados dos dois problemas. O trace referente ao problema da Designação Generalizada apresentou uma quantidade consideravelmente menor de acessos à memória, além de possuir taxas de acerto superiores às obtidas no problema da Conexão

de Circuitos. Entretanto, a quantidade de acessos, por si só, não é suficiente para explicar essa diferença, uma vez que as características e os padrões de reutilização das páginas também influenciam diretamente o comportamento da TLB. Assim, os resultados reforçam a importância da localidade dos acessos à memória para o desempenho desse mecanismo.

REFERÊNCIAS

- [1] A. S. Tanenbaum and H. Bos, *Sistemas Operacionais Modernos*, 4th ed. São Paulo: Pearson, 2016.
- [2] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proceedings of the 2007 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2007, pp. 89–100.